

- i. Dot Product: Eq (1) can be represented by dot product.

$$X \cdot W = \sum_{i=1}^n x_i w_i = x_1 w_1 + x_2 w_2 + x_3 w_3 + \dots + x_n w_n$$
- ii. Slope: Of a linear equation is rate of change of output with respect to input- $\frac{\partial y}{\partial x}$
- iii. Derivative: Is used to find slope of a curve- $\frac{dy}{dx}$
- iv. Chain Rule: Is used to find derivative of composition of two or more functions.

$$\frac{dy}{dx} = \frac{dy}{dz} \cdot \frac{dz}{dx}$$
- v. Partial derivative: When your function depends upon several variables, you will use partial derivative instead of Derivative to find slope- $\frac{\partial y}{\partial x}$
- vi. Matrix transpose: Converts rows into columns
- vii. Mean Square Error: Measures difference between desired output & actual output

$$MSE = \sum_{i=1}^n \frac{1}{2n} (d_i - z_i)^2$$
 Where, d_i is desired output & z_i is actual output.

Box 1: Math basics

1. Sigmoid Function, $\sigma(x) = \frac{1}{1+e^{-x}}$
2. Derivative of Sigmoid Function $\sigma'(x) = x(1-x)$
3. Performance function(MSE), $P = \sum_{i=1}^n \frac{1}{2n} (d_i - z_i)^2$
4. Derivative of Performance Function w.r.t w_1 .

$$\frac{\partial P}{\partial w_1} = \frac{\partial P}{\partial z} \cdot \frac{\partial z}{\partial x} \cdot \frac{\partial x}{\partial w_1} = (z-d) \cdot x(1-x) \cdot y$$
5. Derivative of Performance Function w.r.t w_1 .

$$\frac{\partial P}{\partial w_1} = \frac{\partial P}{\partial z} \cdot \frac{\partial z}{\partial x} \cdot \frac{\partial x}{\partial w_1} = (z-d) \cdot x(1-x) \cdot w_2 \cdot y(1-y) \cdot x$$

Box 2: Equations for DNNs

Sigmoid function1 in layer L1.

2. The output of the Sigmoid function is y , which acts as input to the output layer.
3. Again, randomly initialise weights w_2 . The dot product of y and weights w_2 produces P_2 , which acts as input to the Sigmoid function2 in layer L2.
4. The output of Sigmoid function2 is z , which is our actual output. Give this output to the performance function P along with the desired output to produce an error.
5. Minimise this error by backpropagating it and using the gradient descent algorithm, i.e., change weights w_1 and w_2 to find the minima of the performance function.
6. When the error is minimum, the actual output will be close to the desired output.

The Python code given below shows the practical implementations:

```
import numpy as np
import matplotlib.pyplot as plt

def nonlin(x,deriv=False):
    if(deriv==True):
        return x*(1-x)           #derivative of the
    Sigmoid function
    return 1/(1+np.exp(-x))      #Sigmoid
function

def show_error(l):
    e=[];e1=[];e2=[];e3=[];e4=[]
```

```
count1=0;count2=1;count3=2;count4=3
for i in l:
    for j in i:
        for k in j:
            e.append(k)

    for i in range(10000):
        e1.append(e[count1]);e2.append(e[count2]);e3.
append(e[count3]);e4.append(e[count4])
        count1+=4;count2+=4;count3+=4;count4+=4

plt.plot(e1,label='input1');plt.
plot(e2,label='input2');plt.plot(e3,label='input3');plt.
plot(e4,label='input4')
plt.xlabel('Iteration',fontsize=17);plt.
ylabel('Error',fontsize=17)
plt.legend();plt.grid(True)
plt.show()
```

```
x = np.array([[0,0,1],           #input dataset
              [0,1,1],
              [1,0,1],
              [1,1,1]])

d = np.array([[0],               #desired output dataset
              [1],
              [1],
              [0]])

np.random.seed(1)
w1 = 2*np.random.random((3,4)) - 1 #Randomly initializing
weights in Layer1
w2 = 2*np.random.random((4,1)) - 1 #Randomly initializing
weights in Layer2
l=[]                                #list

for j in xrange(10000):             #Find slope/Derivative for n
number of steps
    l0 = x
    y = nonlin(np.dot(l0,w1)) #dot product of inputs and
weights are given to Sigmoid function which produces output y
    z = nonlin(np.dot(y,w2)) #dot product of inputs and
weights are given to Sigmoid function which produces output z

    p = z - d                       #Error

    l2_delta = p*nonlin(z,deriv=True)
    l1_error = l2_delta.dot(w2.T)
    l1_delta = l1_error * nonlin(y,deriv=True)

    w2 -= y.T.dot(l2_delta)         #new_weight(w2)=old_
weight(w2)-change in weight(i.e.derivative w.r.t w2)
    w1 -= l0.T.dot(l1_delta)        #new_weight(w1)=old_
weight(w1)-change in weight(i.e.derivative w.r.t w1)
```